

手摸手之实现用户购票责任链验证

购票请求验证

在实际购票业务场景中，用户发起一次购票请求后，购票接口在真正完成创建订单和扣减余票行为前，需要验证当前请求中的参数是否正常请求，或者说是否满足购票情况。

1. 购票请求用户传递的参数是否为空，比如：车次 ID、乘车人、出发站点、到达站点等。
2. 购票请求用户传递的参数是否正确，比如：车次 ID 是否存在、出发和到达站点是否存在等。
3. 需要购票的车次是否满足乘车人的数量，也就是列车对应座位的余量是否充足。
4. 乘客是否已购买当前车次，或者乘客是否已购买当天时间冲突的车次。
5. 可能实际场景中需要验证的还有很多，就不逐一举例了。

对于完成这些前置校验逻辑，示例代码如下：

```
public TicketPurchaseRespDTO purchaseTickets(PurchaseTicketReqDTO requestParam) {  
    // 购票请求用户传递的参数是否为空  
    // 购票请求用户传递的参数是否正确  
    // 需要购票的车次是否满足乘车人的数量  
    // 乘客是否已购买当前车次，或者乘客是否已购买当天时间冲突的车次  
    // .....  
}
```

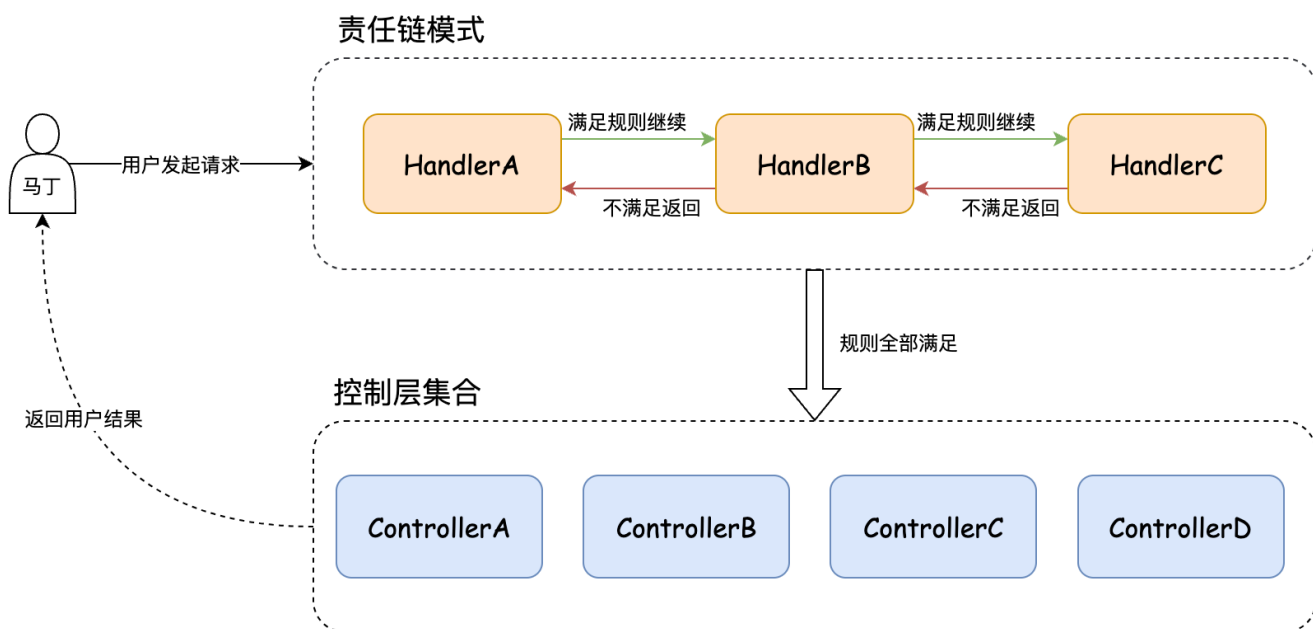
解决前置校验需求需要实现一堆逻辑，常常需要写上几百上千行代码。并且，上面的代码不具备开闭原则，以及代码扩展性，整体来说复杂且臃肿。

为了避免这种坏代码味道，我们可以运用责任链设计模式，对购票验证逻辑进行抽象。

责任链模式

什么是责任链模式

在责任链模式中，多个处理器依次处理同一个请求。一个请求先经过 A 处理器处理，然后再把请求传递给 B 处理器，B 处理器处理完后再传递给 C 处理器，以此类推，形成一个链条，链条上的每个处理器各自承担各自的处理职责。



责任链模式优点

责任链模式的优点在于，它可以动态地添加、删除和调整处理者对象，从而灵活地构建处理链。同时，它也避免了请求发送者和接收者之间的紧耦合，增强了系统的灵活性和可扩展性。

责任链模式重构

定义购票过滤接口

```
package org.opengoofy.index12306.biz.ticketservice.service.handler.ticket.filter.purchase;

import org.opengoofy.index12306.biz.ticketservice.common.enums.TicketChainMarkEnum;
import org.opengoofy.index12306.biz.ticketservice.dto.req.PurchaseTicketReqDTO;
import
org.opengoofy.index12306.framework.starter.designpattern.chain.AbstractChainHandler;

/**
 * 列车购买车票过滤器
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public interface TrainPurchaseTicketChainFilter<T extends PurchaseTicketReqDTO> extends
AbstractChainHandler<PurchaseTicketReqDTO> {
```

```

@Override
default String mark() {
    return TicketChainMarkEnum.TRAIN_PURCHASE_TICKET_FILTER.name();
}
}

```

实现列车购买过滤器实现

```

/**
 * 购票流程过滤器之验证参数必填
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Component
public class TrainPurchaseTicketParamNotNullChainHandler implements
TrainPurchaseTicketChainFilter<PurchaseTicketReqDTO> {

    @Override
    public void handler(PurchaseTicketReqDTO requestParam) {
        // .....
    }

    @Override
    public int getOrder() {
        return 0;
    }
}

/**
 * 购票流程过滤器之验证参数是否有效
 * 验证参数有效这个流程会大量交互缓存，为了优化性能需要使用 Lua。为了方便大家理解流程，这里使用多次调用缓存
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Component
@RequiredArgsConstructor
public class TrainPurchaseTicketParamVerifyChainHandler implements
TrainPurchaseTicketChainFilter<PurchaseTicketReqDTO> {

```

```

private final TrainMapper trainMapper;
private final TrainStationMapper trainStationMapper;
private final DistributedCache distributedCache;

@Override
public void handler(PurchaseTicketReqDTO requestParam) {
    // .....
}

@Override
public int getOrder() {
    return 10;
}
}

/**
 * 购票流程过滤器之验证列车站点库存是否充足
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Component
@RequiredArgsConstructor
public class TrainPurchaseTicketParamStockChainHandler implements
TrainPurchaseTicketChainFilter<PurchaseTicketReqDTO> {

    private final SeatMarginCacheLoader seatMarginCacheLoader;
    private final DistributedCache distributedCache;

    @Override
    public void handler(PurchaseTicketReqDTO requestParam) {
        // .....
    }

    @Override
    public int getOrder() {
        return 20;
    }
}

/**
 * 购票流程过滤器之验证乘客是否重复购买
 *

```

```

* @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
*/
@Component
@RequiredArgsConstructor
public class TrainPurchaseTicketRepeatChainHandler implements
TrainPurchaseTicketChainFilter<PurchaseTicketReqDTO> {

    @Override
    public void handler(PurchaseTicketReqDTO requestParam) {
        // .....
    }

    @Override
    public int getOrder() {
        return 30;
    }
}

```

购票流程使用过滤器

```

private final AbstractChainContext<PurchaseTicketReqDTO>
purchaseTicketAbstractChainContext;

@Override
@Transactional(rollbackFor = Throwable.class)
public TicketPurchaseRespDTO purchaseTickets(PurchaseTicketReqDTO requestParam) {
    // 责任链模式，验证 0：参数必填 1：参数正确性 2：列车车次余量是否充足 3：乘客是否已买当前
    车次等

    purchaseTicketAbstractChainContext.handler(TicketChainMarkEnum.TRAIN_PURCHASE_TICKET_FILTE
R.name(), requestParam);
    // .....
}

```

购票责任链实现原理

运行时获取责任链具体实现类

方法位置:

org.opengooify.index12306.biz.ticket.service.service.impl.TicketServiceImpl#purchaseTickets

```
private final Map<String, List<AbstractChainHandler>> abstractChainHandlerContainer = new
HashMap<>();

/**
 * 责任链组件执行
 *
 * @param mark          责任链组件标识
 * @param requestParam 请求参数
 */
public void handler(String mark, T requestParam) {
    // 通过 mark 获取到本次需要执行的责任链组件
    List<AbstractChainHandler> abstractChainHandlers =
abstractChainHandlerContainer.get(mark);
    // 获取为空抛出异常
    if (CollectionUtils.isEmpty(abstractChainHandlers)) {
        throw new RuntimeException(String.format("[%s] Chain of Responsibility ID is
undefined.", mark));
    }
    // 获取到的责任链组件依次执行
    abstractChainHandlers.forEach(each -> each.handler(requestParam));
}
```

初始化责任链容器

```
// CommandLineRunner: SpringBoot 启动完成后执行的回调函数
public final class AbstractChainContext<T> implements CommandLineRunner {

    // 存储责任链组件实现和责任链业务标识的容器
    // 比如: Key: 购票验证过滤器 Val: HanlderA、HanlderB、HanlderC、.....
    private final Map<String, List<AbstractChainHandler>> abstractChainHandlerContainer =
new HashMap<>();
```

```

@Override
public void run(String... args) throws Exception {
    // 调用 SpringIOC 工厂获取 AbstractChainHandler 接口类型的 Bean
    Map<String, AbstractChainHandler> chainFilterMap = ApplicationContextHolder
        .getBeansOfType(AbstractChainHandler.class);
    chainFilterMap.forEach((beanName, bean) -> {
        // 获取 mark（责任链业务标识）的处理器集合
        List<AbstractChainHandler> abstractChainHandlers =
abstractChainHandlerContainer.get(bean.mark());
        // 如果不存在则创建一个集合
        if (CollectionUtils.isEmpty(abstractChainHandlers)) {
            abstractChainHandlers = new ArrayList();
        }
        // 添加到处理器集合中
        abstractChainHandlers.add(bean);
        // 对处理器集合执行顺序进行排序
        List<AbstractChainHandler> actualAbstractChainHandlers =
abstractChainHandlers.stream()
            .sorted(Comparator.comparing(Ordered::getOrder))
            .collect(Collectors.toList());
        // 存入容器等待被运行时调用
        abstractChainHandlerContainer.put(bean.mark(), actualAbstractChainHandlers);
    });
}
}

```

更新: 2023-08-23 15:11:34

原文: <<https://www.yuque.com/magestack/12306/ggg2txzbfqfqp6tm>>